MTP 与DSpark（V4.1-Flash）

1. MTP：原生串行自回归草稿

MTP 不引入独立草稿模型，在预训练阶段让主模型附带一个轻量Transformer 块，在主干输出的hidden 上继续串行预测后续token。

机制：串行递推

设batch 为B，hidden 为H，词表为V。主模型自回归解码到第t 步、经过全部主干层后，位置t的hidden $h_t \in \mathbb{R}^{B\times H}$ 已经产生。MTP Head 复用这份hidden 做串行递推：

$$h'_{t+1} = \text{MTP_Block}_1(h_t, E(x_t)), \tag{1}$$
$$h'_{t+2} = \text{MTP_Block}_2(h'_{t+1}, E(x_{t+1})), \tag{2}$$

每步将前一步hidden 与前一预测token 的嵌入投影融合，输出下一位置logits $\in \mathbb{R}^{B\times V}$。由于第$k+1$步的输入依赖第$k$ 步采样出的token，L 步无法在单个非因果前向中并行展开，实际执行L 次顺序的$[B,H] \to [B,V]$ 前向。V4-Flash 原生MTP 通常只多预测1–2 个token，原因正在于串行延迟会抵消草稿延长的收益。

分叉结构

“主模型已经算出h_t”不意味着主模型为MTP 额外做了一次前向：h_t 是正常自回归解码本来就产出的主干输出。MTP Head 是挂在这份hidden 上的额外分支，不在主模型正常输出的必经路径上。

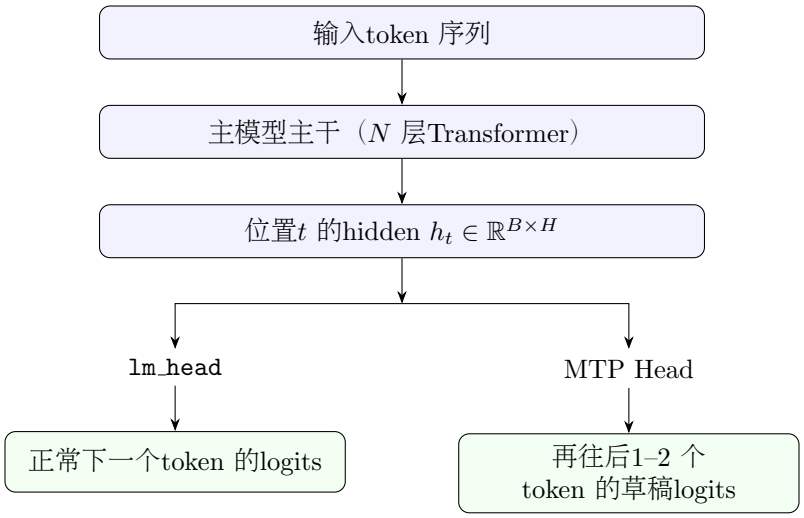


Figure 1: 主干输出h_t 分叉：`lm_head` 给出正常下一个token；MTP Head 在同一hidden 上串行递推草稿。主模型正常输出不经过MTP Head。

草稿验证

MTP Head 只产生草稿logits，不确认草稿token。验证时把原始上下文与保留的K 个草稿token 拼成一次批量输入，主模型一次并行前向，接受最长匹配前缀，在第一个拒绝位置重采样正确token，作为下一轮anchor。

V4-Flash 的原生MTP 头是主模型联合预训练的一个额外Transformer 层（`num_nextn_predict_layers = 1`），复用`DeepseekV4DecoderLayer`，携带MLA 注意力与MoE，draft 路径的shape 与主干decode

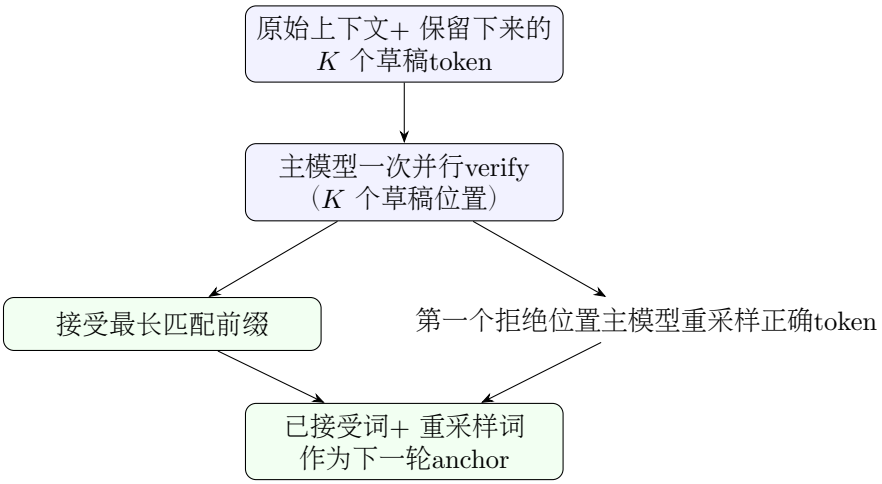


Figure 2: MTP 草稿的并行验证：草稿不逐token 串行回主干，而是与上下文拼成一次批量输入并行verify。

高度同构。权重从target checkpoint 的`mtp.0.*` 读取，`embed_tokens` 与`lm_head` 共享别名，draft 侧不持有独立embed/head 权重。MTP 的加速收益与主模型强绑定：每一步都是主模型权重内部长出来的前向。

特点

- 因果保真：逐token 串行预测，第2 个草稿的预测条件于第1 个草稿的采样结果。
- 训练代价：必须与主模型联合预训练，是模型权重的一部分。
- 草稿长度短（通常2–3 个）：草稿过长时串行计算本身的延迟会抵消加速收益。
- 验证方式：草稿回到主模型做一次并行verify，在第一个拒绝位置重采样；hidden 只在主干输出处分叉一次，验证是另一次主模型前向。

2. DSpark（V4.1-Flash）：并行骨架+ 顺序精修+ 置信度调度

DSpark 外挂草稿模块：草稿骨干并行粗排，轻量Markov 头顺序注入前缀依赖，置信度调度器按硬件吞吐动态截断验证长度。

阶段1：两阶段草稿生成

并行阶段（DFlash）。设block 大小为N，输入一次铺开N 个query 位置，草稿骨干做一次非因果前向：

$$H_{\text{draft}} \in \mathbb{R}^{B\times N\times H}, \qquad U \in \mathbb{R}^{B\times N\times V},$$

U_k 是第k 个位置的基础logits。块内位置互相可见，墙钟时间近似与N 无关，使草稿块可以拉到5–7 而不被串行延迟吃掉。

顺序精修（VanillaMarkov 头）。并行logits 的位置之间没有依赖，块后段容易前后矛盾。DSpark 默认使用无状态VanillaMarkov 头，在U_k 上叠加一个低秩token-to-token 偏置：

$$e_{k-1} = W_1[x_{k-1}], \tag{3}$$
$$\text{bias}_k = W_2 e_{k-1}, \tag{4}$$
$$p_k = \text{softmax}(U_k + \text{bias}_k), \tag{5}$$
$$x_k \sim p_k, \tag{6}$$

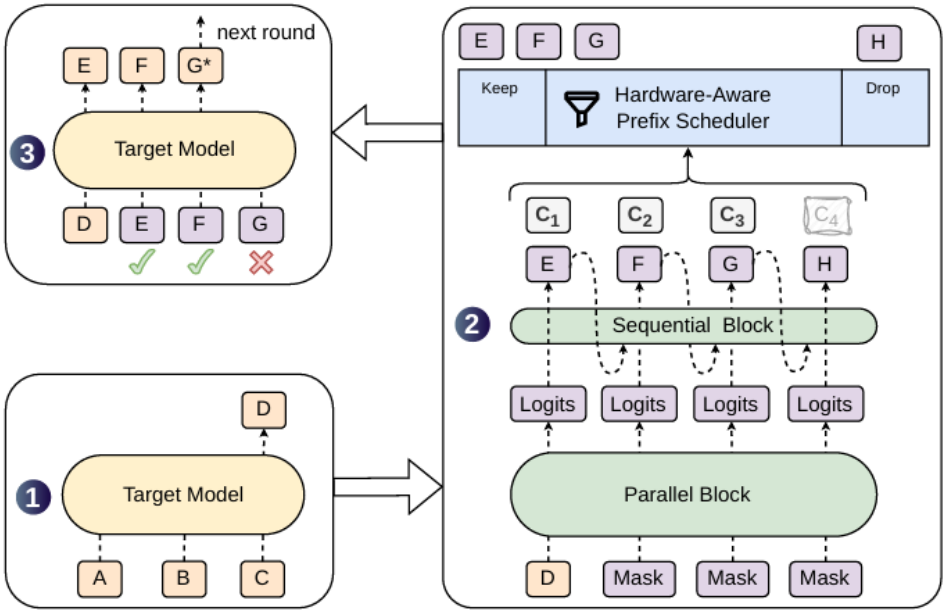


Figure 3: DSpark 推测解码流程：并行粗排、顺序精修、置信度调度、目标模型并行验证。

其中$W_1 \in \mathbb{R}^{V\times r}$，$W_2 \in \mathbb{R}^{V\times r}$，低秩$r$（典型配置为512 或256）。它不重跑Transformer 前向，shape 流为$[B,V] \to [B,r] \to [B,V]$，每步是表查找与投影级别开销。VanillaMarkov 只依赖前一采样token，不携带跨步隐状态；代码中保留的`GatedMarkov` 与`RNNHead` 可引入更长状态，但不是V4.1-Flash 的默认路径。这也是DSpark 与Domino（GRU 携带递归前缀状态）的关键区别。

阶段2：硬件感知前缀调度

置信度头为每个位置输出条件接受概率，得到

$$C \in \mathbb{R}^{B\times N}.$$

硬件感知前缀调度器不消费完整logits，只消费C，结合profiled 的SPS(B) 吞吐曲线（固定CUDA-graph batch size 上采样，稠密化为$O(1)$ 查表），为每个请求决定实际送verify 的前缀长度$K_r \le N$：系统空闲时验证更长前缀；高并发时截掉尾部低置信度草稿，只把高概率前缀送进主模型。调度器按存活概率从高到低单调处理候选，只有当前缀全局价值最高时才延长验证。部分V4-Flash 生产配置中可观察到batch 1–3 时$K = 5$、batch 4–256 时$K = 0$；V4.1-Flash 的默认草稿块$N = 5$，vLLM 等栈常见配置可到7。

阶段3：目标模型并行验证

主模型接收原始上下文与保留的K_r 个草稿，把草稿位置与上下文拼成一次批量并行verify：接受最长匹配前缀，在第一个拒绝位置重采样正确token，已接受词与重采样词作为下一轮anchor。

V4.1-Flash 执行时序

V4.1-Flash 采用CED（Causal Encoder-Decoder）架构：40 层因果Transformer 切分为前20 层Encoder 和后20 层Decoder，Prefill 激活约8B，Decode 激活约16B。DSpark 严格在Prefill（含Encoder 全量计算）结束后才被激活；验证时草稿token 必须流经Encoder 层（复用上下文KV）获取全局表征，再进入Decoder 做因果并行验证。

一次完整生命周期如下：

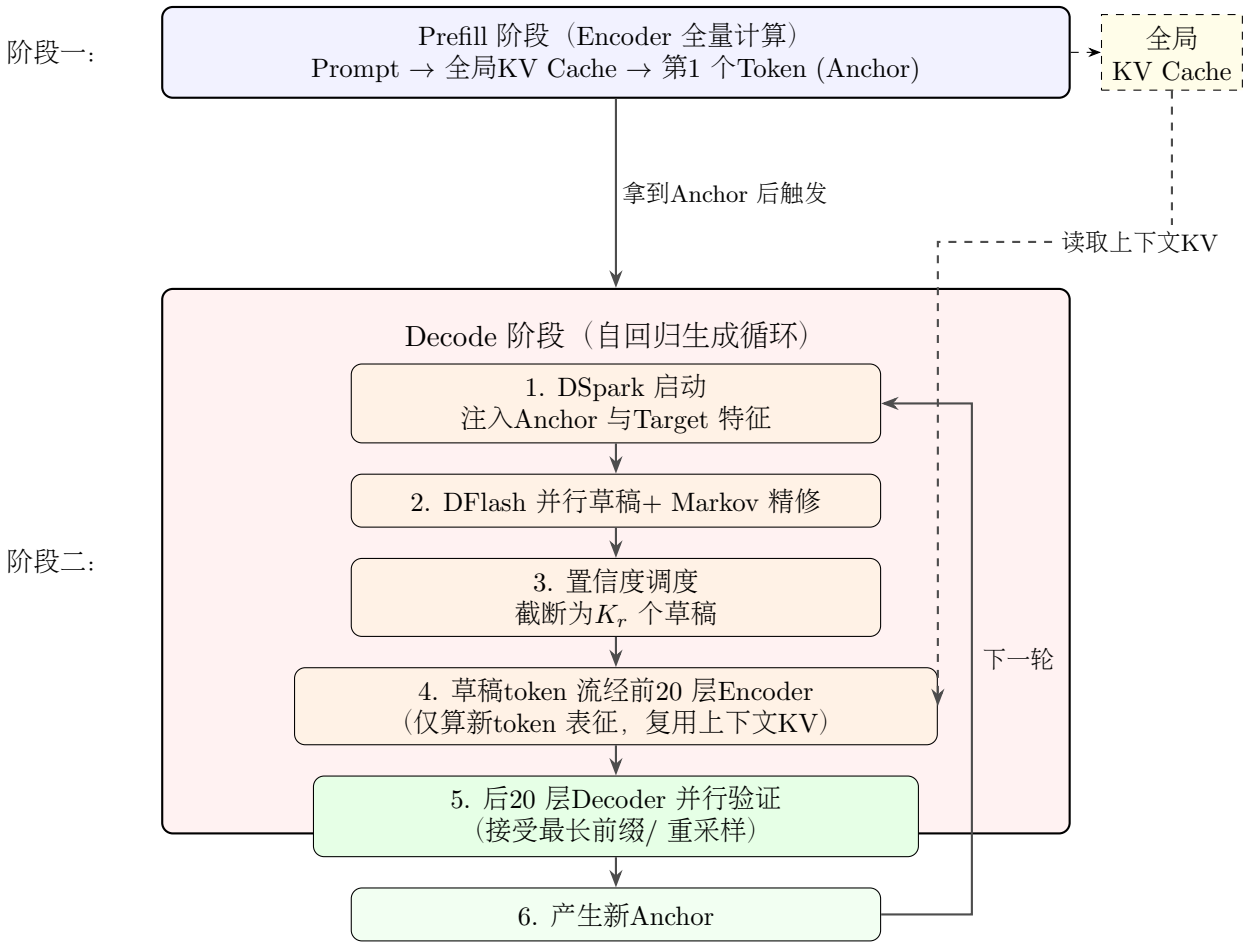


Figure 4: V4.1-Flash 中DSpark 的执行时序。Prefill 由前20 层Encoder 完成全量计算并产出首个Anchor；DSpark 随后激活。验证时草稿token 先流入Encoder 复用上下文KV 获取表征，再进入Decoder 并行验证。

1. Prefill（Encoder 全量）：前20 层Encoder 处理全部上下文，构建全局KV Cache；DSpark 休眠。
2. 触发：Prefill 结束后输出第一个Anchor Token，DSpark 启动。
3. 打草稿：DFlash 并行吐出候选，Markov 头精修，置信度调度截断为K_r 个。
4. 草稿流经Encoder：K_r 个草稿token 作为新输入送入前20 层Encoder，复用已缓存的上下文KV，只为新增token 计算表征并写入KV Cache。
5. Decoder 并行验证：带全局表征的草稿进入后20 层Decoder，因果掩码下一次前向并行verify，接受最长前缀，在拒绝位置重采样。
6. 迭代：接受词与重采样词合并为新Anchor，进入下一轮，直到EOS。

V4.1-Flash 的DSpark 原生内嵌在checkpoint 中：`num_nextn_predict_layers=3`，草稿器由3 个Transformer 块组成。DFlash 骨干与顺序Markov 采样可被FULL CUDA graph 覆盖，整步host/kernel launch 开销显著降低。

3. 重采样：拒绝位置回到正确轨迹

无论是MTP 的固定长度验证，还是DSpark 经置信度调度后的动态前缀验证，草稿token 都不会被直接信任。验证阶段把“原始上下文+ 保留下来的草稿前缀”拼成一次批量输入，由主模型一次并行前向，得到每个草稿位置在目标模型下的条件分布。

设当前上下文为c，草稿模型（Draft）在位置i 生成草稿token $\tilde{x}_i$ 时的分布为$q_i(\cdot)$，目标模型（Target）在相同前缀下的分布为$p_i(\cdot)$。推测解码为了严格保证最终输出分布与目标模型完全一致（无损），采用拒绝采样（Rejection Sampling）：

1. 接受概率：以概率$r_i = \min\left(1, \frac{p_i(\tilde{x}_i)}{q_i(\tilde{x}_i)}\right)$ 接受草稿$\tilde{x}_i$。具体实现是采样一个均匀随机数$u \sim U(0,1)$，若$u \le r_i$ 则接受，否则拒绝。
2. 残差重采样：如果第一个被拒绝的位置在$\ell+1$，则丢弃$\tilde{x}_{\ell+1}$ 及其后的所有草稿，并从修正后的残差分布中重新采样：

$$p'(\cdot) = \frac{\max(0, p_{\ell+1}(\cdot) - q_{\ell+1}(\cdot))}{\sum_x \max(0, p_{\ell+1}(x) - q_{\ell+1}(x))}.$$

如果采用贪心验证（Greedy）（即温度$T=0$ 的确定性解码），则不需要计算接受概率：只要p_i 的argmax 与$\tilde{x}_i$ 一致就接受，一旦不一致直接拒绝并在该位置取p_i 的argmax 作为正确token。

一个具体的计算示例

设当前上下文为$c=(A,B,C)$。假设词表为$V=\{A,B,C,D\}$，DSpark 保留的草稿token 为$\tilde{x}_{1:3}=(D,A,B)$，且草稿模型在这三个位置输出的分布分别为q_1,q_2,q_3。

主模型将上下文与草稿拼接成(A,B,C,D,A,B)，一次并行前向，得到目标模型分布p_1,p_2,p_3。

第1 个草稿位置（验证$\tilde{x}_1=D$）：

$$q_1 : D=0.8, A=0.1, B=0.1, C=0.0$$
$$p_1 : D=0.85, A=0.05, B=0.05, C=0.05$$

接受概率$r_1=\min(1, 0.85/0.8)=1$。草稿D 必然被接受。

第2 个草稿位置（验证$\tilde{x}_2 = A$）：

$$q_2 : A = 0.5, B = 0.3, C = 0.1, D = 0.1$$
$$p_2 : A = 0.6, B = 0.2, C = 0.1, D = 0.1$$

接受概率$r_2 = \min(1, 0.6/0.5) = 1$。草稿$A$ 必然被接受。

第3 个草稿位置（验证$\tilde{x}_3 = B$）：

$$q_3 : B = 0.6, C = 0.2, D = 0.1, A = 0.1$$
$$p_3 : D = 0.7, B = 0.15, A = 0.1, C = 0.05$$

接受概率$r_3 = \min(1, 0.15/0.6) = 0.25$。假设采样随机数$u = 0.8 > 0.25$，因此草稿$B$ 被拒绝，这里就是第一个拒绝位置。

执行重采样：此时丢弃B，计算残差分布p'：

$$p'(A) \propto \max(0, 0.1 - 0.1) = 0$$
$$p'(B) \propto \max(0, 0.15 - 0.6) = 0$$
$$p'(C) \propto \max(0, 0.05 - 0.2) = 0$$
$$p'(D) \propto \max(0, 0.7 - 0.1) = 0.6$$

归一化后得到$p'(D) = 1.0$。从该分布采样，必然得到D。则下一轮anchor 变为：

$$a = (A, B, C, D, A, D).$$

注：如果采用贪心验证，第*3* 步发现p_3 的$argmax$ 是D 而非草稿B，直接拒绝并取D，结果相同。

4. MTP 与DSpark（V4.1-Flash）对比

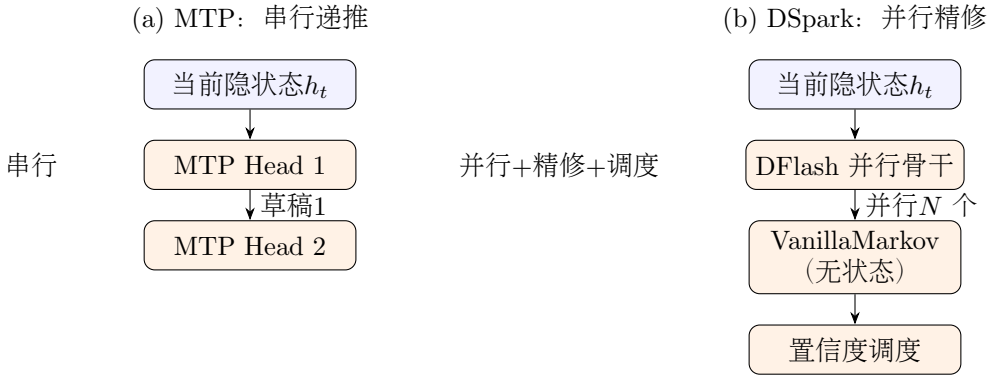


Figure 5: MTP 串行递推vs DSpark 并行精修。DSpark 默认无状态VanillaMarkov 头，不携带跨步隐状态。

维度	MTP	DSpark（V4.1-Flash）
生成机制	串行自回归。每步完整Transformer 前向，严格保持因果性。	并行粗排（DFlash）+ 无状态低秩精修（Markov）。一阶近似因果。
调度策略	固定长度验证（通常2–3 个）。	置信度+ 吞吐曲线动态截断$K_r \leq N$；高并发可截断至0。
依赖与训练	强绑定主模型，需联合预训练；复用主干MLA+MoE。	解耦外挂，主模型冻结；V4.1-Flash 原生内嵌3 个草稿层。
草稿长度	短（2–3 个）。	较长（原生5，最大可至7）。
工程性能	串行多步难以整步CUDA graph 捕获。	并行骨干+ 轻量采样可被FULL CUDA graph 覆盖。

Table 1: MTP 与DSpark（V4.1-Flash）核心异同。